

Evaluation of ML Algorithms in Network-Based Intrusion Detection Using a Progressive Dataset — Reproduction and Critical Evaluation

Course: Data Science Methods in Cybersecurity — Final Project

Evaluated source: Chua, T.-H. & Salam, M. I. (2023). *Evaluation of Machine Learning Algorithms in Network-Based Intrusion Detection Using Progressive Dataset*. Symmetry, 15(6), 1251. <https://doi.org/10.3390/sym15061251>

Authors' repository: <https://github.com/tuanhong3498/Evaluation-of-Machine-Learning-Algorithm-in-Network-Based-Intrusion-Detection-System>

Project repository: <https://github.com/Rosette28/data-science-cyber-final-project>

1. Summary of the Source

The problem being addressed. Machine-learning-based network intrusion detection systems (IDS) are almost always evaluated by training and testing on data drawn from the same time window and the same capture environment. This produces optimistic accuracy numbers (routinely 99%+) that say nothing about how the model behaves once deployed, when attack tooling, traffic composition, and network environments have changed. The paper asks: *do models that look excellent in-distribution actually keep working on future traffic?*

Why the problem is important. An IDS is deployed precisely to catch *future* attacks. If published accuracy figures are artifacts of same-distribution evaluation, practitioners may deploy models that silently stop detecting attacks months after training — a false sense of security that is worse than no IDS at all. The gap between benchmark performance and deployment performance is one of the central unsolved problems in applied security ML.

The proposed solution. *Progressive evaluation:* train models on an older dataset and test them on a newer, temporally disjoint one collected in a different environment. Concretely, the paper trains on CIC-IDS2017 (July 2017) and tests on CSE-CIC-IDS2018 (February–March 2018) — an ~8-month gap with zero temporal overlap. The size of the performance drop is treated as a measure of how much each model "overfits" the training distribution. (A secondary experiment uses the LUFlow dataset; that experiment is out of scope for this reproduction.)

The dataset used. CIC-IDS2017 (~2.8M flows, 78 CICFlowMeter features, benign traffic plus 15 attack types: DoS/DDoS variants, brute force, web attacks, port scanning, botnet, infiltration) and CSE-CIC-IDS2018 (~16M flows, similar feature schema, partially different attack mix). Both are lab-generated captures from the Canadian Institute for Cybersecurity, processed with CICFlowMeter into per-flow statistical features.

The model / methodology employed. Six classifiers — Decision Tree (DT), Random Forest (RF), SVM (RBF kernel), Gaussian Naive Bayes (NB), ANN (1-hidden-layer MLP), and DNN (3-hidden-layer MLP), all in scikit-learn. Pipeline: clean (drop inf/NaN, duplicate column), collapse all attack labels to a binary `malicious` label, downsample benign to a **1:1 ratio** (the "symmetry" of the paper's title), select 11 features via a two-stage process (RF importance ranking, then a brute-force add-one-feature accuracy loop), tune hyperparameters with k=5 GridSearchCV, then evaluate in-distribution and on the progressive test set using Accuracy, Precision, Recall, and F1. The headline result: DT/RF achieve ~99.6–99.7% in-distribution but collapse on 2018 data, while SVM (and, the authors claim, ANN) retain the most performance; the authors conclude SVM/ANN "resist overfitting" and recommend ANN for long-term deployment.

2. Critical Evaluation

The main claims made by the author:

1. Progressive evaluation exposes generalisation failure that single-dataset cross-validation misses.
2. SVM and ANN are the most resistant to the progressive performance drop; ANN is the best choice for long-term deployment.
3. DT and RF overfit the most (near-perfect in-distribution, largest drop).
4. The cross-dataset drop is caused by **overfitting**.
5. 1:1 class balancing ("symmetry") is required for fair evaluation.
6. NB underperforms because its feature-independence assumption is violated and the selected features do not suit it.
7. 11 features are optimal for the CIC datasets.

Are the claims supported by the evidence? Partially — and reproduction (Section 5) directly contradicts two of them.

- **Claim 1 — supported, and reproduced.** This is the paper's real contribution. Every model I trained lost 14–44 accuracy points between in-distribution CV and the 2018 test under matched class balance. Single-dataset evaluation would have reported 96–99.7% for all six models and ranked them almost in reverse order of their actual out-of-distribution merit.
- **Claim 2 — half supported.** SVM's top ranking reproduces cleanly under every protocol I ran (best recall 0.745, best MCC 0.604, smallest matched-balance drop –0.139). **ANN's second place does not reproduce.** In run DNN beats ANN on every generalisation metric — MCC 0.53 vs. 0.21, ROC-AUC 0.91 vs. 0.82, recall 0.58 vs. 0.27 — and under the paper's own 1:1-balanced test protocol, DNN scores 0.757 accuracy vs. ANN's 0.590, a 17-point reversal of the paper's 5-point ANN advantage. Since the paper builds a deployment recommendation ("use ANN") on this ranking, the reversal matters practically, not just numerically.
- **Claim 3 — the observation reproduces; the interpretation does not** (see Claim 4). DT and RF do drop the most (matched-balance drops of –0.444 and –0.427) and their recall collapses to ~14% on 2018 data.

- **Claim 4 — not supported.** Classical overfitting means fitting training noise, which shows up *within* the training distribution: high variance across CV folds or a train-vs-validation gap. I measured this directly: every model's k=5 CV standard deviation is $\leq 0.32\%$, and DT/RF — the models the paper blames most — have the *lowest* ($\pm 0.07\%$ and $\pm 0.06\%$). The models are stable and accurate on unseen folds of 2017 data and fail only when the distribution itself changes. That is **concept drift / dataset shift**, not overfitting. The distinction is not pedantic: overfitting is mitigated by regularisation or more same-distribution data, while drift requires retraining on newer data or drift-robust features — the paper's framing points practitioners toward the wrong fix. The paper never reports CV variance or any within-distribution stability check, so its own evidence cannot distinguish the two explanations.
- **Claim 5 — misleading as stated.** 1:1 downsampling is one pragmatic choice among several (class weights, SMOTE, threshold tuning), not a methodological principle; it discards 63.8% of the training data (170,167 of 266,739 rows in run), and — critically — the paper *also* balances the test set to 1:1, so its reported metrics describe an artificial 50/50 world. When I re-evaluated the same models at real prevalence (84.1% benign), accuracy figures moved by up to 32 points, and the model ranking by accuracy changed. Reporting only the balanced numbers hides exactly the deployment-relevant behaviour the paper claims to measure.
- **Claim 6 — mechanism confirmed, attribution untested by the authors.** The paper asserts the independence-assumption violation without testing it. feature-selection curve provides the direct evidence the paper lacks: NB's CV accuracy falls off a cliff ($0.841 \rightarrow 0.723$) at the exact step where a feature nearly perfectly correlated with an already-selected one enters the set. The paper's secondary claim (that feature selection didn't suit NB) is never tested with NB-specific feature sets, by them or by us.
- **Claim 7 — weakly supported.** accuracy-vs-number-of-features curve shows no clean elbow at 11; a plateau-detection rule applied to own curve suggests ~ 15 . "11 is optimal" is a defensible reading of their curve, not a robust finding; the selected set is also seed- and subsample-dependent (only 7 of top-11 match theirs).

Is the evaluation methodology appropriate? The core design (temporal train/test separation) is appropriate and ahead of common practice. Four weaknesses limit it: (a) the metric set — Accuracy/Precision/Recall/F1 only, with no base-rate-robust metric (MCC) or threshold-independent metric (ROC-AUC), on a task where base-rate effects turn out to dominate the accuracy column; (b) balancing the *test* set, which converts a deployment question into a benchmark question; (c) no statistical significance testing — all results are single-run point estimates with no seed variance, so differences of a few points (including the paper's ANN-over-DNN margin) cannot be distinguished from sampling noise; (d) no per-attack-type breakdown, which error analysis shows is where the real story is (two attack types drive most of the aggregate drop).

Possible weaknesses or limitations. Both datasets are lab simulations from the same institution, so even the "progressive" gap understates real-world drift in some respects (same flow-feature extractor, similar traffic generators) while overstating it in others (attack mixes differ drastically — PortScan is 26% of 2017 attacks and absent from 2018; XSS grows 150×). The 10% subsampling is

not discussed for its effect on rare classes (Heartbleed $n=1$, Infiltration $n=6$ in train sample are unlearnable). Manual hyperparameter transcription between the authors' notebooks is a reproducibility hazard.

Are the conclusions justified? The headline conclusion — progressive evaluation is necessary, and SVM generalises best — is justified and reproduces. The causal explanation (overfitting) and the specific deployment recommendation (ANN) are not justified by their evidence, and reproduction actively contradicts the latter.

3. Feature Engineering Analysis

Was feature engineering performed? Yes, in both the original work and this reproduction. The original applies cleaning, binary relabelling, 1:1 downsampling, and a two-stage feature selection (no scaling for tree models; scaling for SVM/ANN/DNN). I reproduce all of it and add derived features, systematic redundancy analysis, and explicit justification for each transformation.

Which features were used. 78 CICFlowMeter per-flow statistics, falling into five semantic groups: packet-length statistics (what is being sent — floods use fixed sizes, exfiltration sends large payloads), packet/byte counts and rates (volume and asymmetry — DoS is extremely unidirectional), inter-arrival times (timing — scans and floods have tiny, regular IATs), TCP flag counts and initial window sizes (connection behaviour — SYN-without-ACK, non-default window sizes fingerprint tools), and port/header/idle statistics. After hygiene cleaning, 66 base features remain; the final model set is 11.

Transformations applied, why, and their measured effect:

- **Cleaning (both):** strip column-name whitespace; drop the duplicated `Fwd Header Length.1` column; replace $\pm\text{inf}$ with NaN and drop ($\sim 0.05\%$ train / 0.63% test rows). *Why:* inf/NaN in rate features (division by zero-duration flows) break scaling and sklearn estimators. *Effect:* purely enabling; no information loss of note.
- **Our additions at load time:** float64→float32 downcast (halves RAM, error $< 1e-7$, needed for free Colab); row deduplication (removes 5.2% train / 14.3% test exact-duplicate rows — repeated identical attack-script flows add no information and bias class statistics); dropping 10 constant columns (the CICFlowMeter bulk-transfer heuristics are all-zero in these captures — zero-variance features are pure noise dimensions). *Evidence:* shapes $266,739 \times 67$ train / $106,906 \times 67$ test after cleaning; no metric can be harmed by removing zero-variance columns or exact duplicates.
- **Encoding (both: none needed — with one justified decision).** All 66 features are numeric. The only genuine encoding question is `Destination Port` ($8,927$ unique integer values). One-hot is infeasible ($65,536$ levels), label encoding is arbitrary, range-binning loses within-bin discrimination; I (like the paper) keep it as an integer: tree models learn meaningful splits such as `port  $\leq 1023$` , and after standardisation it enters SVM/ANN on equal scale. Limitation: port-specific patterns are exactly the kind of signal that fails to transfer across datasets.
- **Scaling (both):** StandardScaler, fitted **only on the balanced training set** and applied to the real-prevalence copy and the test set (fitting on test would leak the test distribution). *Why:* the

RBF-SVM is distance-based (`Destination Port` at 0–65,535 would drown flag counts at 0–5) and MLP training converges better on zero-mean unit-variance inputs; DT/RF/NB are scale-invariant, but a uniform pipeline is simpler. *Effect*: enabling for SVM/ANN/DNN; verified $\text{mean} \approx 0$, $\text{std} \approx 1$ on the training matrix.

- **Aggregation / feature creation (mine)**: four derived flow features with cyber rationale. Two entered the final top-11: `feat_win_ratio` (forward/backward initial TCP window ratio — attack tools use non-OS-default windows; ranked **3rd** of 70 by RF importance, and its addition produces the single largest jump in the whole selection curve, e.g. LinearSVC 0.684→0.868) and `feat_bytes_per_pkt` (rank 11). `feat_pkt_len_range` ranked 16th. `feat_bwd_fwd_ratio` failed instructively: it separates attack *types* cleanly (Slowhttptest = 0.0, FTP-Patator = 1.667) but after binary collapse the per-type signals average to a median of exactly 1.0 — identical to benign. This is direct evidence that the paper's binary relabelling destroys usable signal.
- **Feature selection (both)**: RF Gini-importance ranking, then a brute-force add-one loop (LinearSVC/NB/MLP, 3-fold CV on a 10k subsample). I select 11 features to match the paper; **7 of 11 overlap with theirs**, differences attributable to seed, subsample, and derived features. *Effect*: the curve shows ~0.88 (LinearSVC) and ~0.94 (MLP) CV accuracy is reached by $n=11$ vs. ~0.92/0.955 at $n=20$ — 11 features retain most of the discriminative power at a sixth of the dimensionality.
- **Dimensionality reduction**: PCA deliberately not used — named features are needed for the security interpretation (which attack types map to which feature ranges), and tree models don't benefit.

Is there redundancy in the system? How to spot it and tackle it. Yes, substantial, and it is one of main findings. I detected it three independent ways: (1) **Spearman scan** — 99 feature pairs with $|\rho| > 0.90$, ten of them exactly 1.000 (e.g. the four `Subflow *` columns duplicate the `Total *` columns; `Avg Segment Size` = `Packet Length Mean`; `Variance` = `Std2`); (2) **RF importance distribution** — importance mass is split roughly evenly across identical twins, which proves that *RF-importance top-N selection does not remove exact duplicates* (an $r=1.000$ pair, `Avg Bwd Segment Size` and `Bwd Packet Length Mean`, both landed in final 11 — and in the paper's final 11 too); (3) **VIF on the final set** — $\text{VIF} = \infty$ for that pair, 780/670/343 for other size features, confirming severe multicollinearity. *How to tackle*: correlation-filter (drop one of each $|\rho| > 0.95$ pair) *before* importance ranking; VIF-based iterative pruning where model assumptions require near-independence (NB, logistic regression); PCA where interpretability is expendable. The consequence of not tackling it is measurable: NB — the only model whose assumptions the redundancy violates — is 12+ points below every other model in-distribution and collapses to below-chance accuracy (0.484) on the balanced progressive test.

Was the feature engineering meaningful (mathematical + cybersecurity view)? Mostly yes. Mathematically, the selected features are dominated by packet-size statistics whose heavy-tailed, multimodal distributions genuinely separate flooding/scanning behaviour from interactive traffic, and the selection curve empirically validates each addition. From a cybersecurity standpoint the features encode real attack mechanics (window-size fingerprints, unidirectionality of floods, tiny probe packets). Two caveats: the redundancy just described was left in by the authors' method, and

packet-size dominance is itself a fragility — attack tools can trivially pad packet sizes, and error analysis shows attack types whose sizes resemble benign traffic (DDoS-LOIC-UDP, much of Brute Force-XSS) are missed by *all* six models.

Additional features that could improve performance. HTTP/application-layer features (URI entropy, header anomalies) — required to catch the XSS flows that are indistinguishable from benign traffic in pure flow statistics; per-host aggregates over time windows (flows per source per minute, unique destination ports per source — classic scan detectors); TLS metadata (JA3 fingerprints); sequence features across consecutive flows (n-gram of flow sizes); and drift-robust normalised ratios in place of absolute byte counts.

4. Reproducibility Analysis

Can the code be executed successfully? The authors' repository contains five sequential Jupyter notebooks (preprocessing → feature selection → hyperparameter optimisation → evaluation ×2). The code is readable and the pipeline logic is recoverable from it — entire reproduction was built by mirroring those notebooks. However, running them unmodified today is not straightforward: they were written for Python 3.8.8 / scikit-learn 0.24.1, hard-code local file paths to the full multi-gigabyte raw CSVs, and assume the full datasets fit in memory (the full CSE-CIC-IDS2018 is ~16M rows). Executing on free-tier hardware requires reworking the data loading (we implemented chunked reading with 10% subsampling).

Are all required files and dependencies available? Code: yes, public on GitHub. Data: yes, but not bundled and not perfectly uniform across sources — CIC-IDS2017 and CSE-CIC-IDS2018 exist in several public variants (raw PCAPs, the official UNB pre-processed CSVs, and multiple independent Kaggle re-uploads), which do not all share the same column names or label encoding. This project uses two public Kaggle mirrors ([chethuhn/network-intrusion-dataset](#) for CIC-IDS2017, [ekkykharismadhany/csecicids2018-cleaned](#) for CSE-CIC-IDS2018) rather than the original UNB/AWS distributions, because the official downloads are multi-GB and gated behind slower institutional mirrors — a practical necessity the paper itself does not have to navigate, since its authors presumably used the primary source directly. This is itself a small but real reproducibility finding: the paper does not pin an exact file hash or distribution variant, so two students following its stated data source could end up training on measurably different CSVs. Concretely, CIC-IDS2017 mirror carries a UTF-8-mangled en-dash in three `Web Attack` label strings, and CSE-CIC-IDS2018 mirror encodes `Label1` as integers rather than strings — both handled explicitly in `src/data_loading.py`, but neither is documented anywhere in the paper as a preprocessing step a re-implementer would need to anticipate. Dependency versions are stated in the paper but there is no `requirements.txt` lock in the original repo (ours is pinned).

Do hidden preprocessing steps exist? Yes, three notable ones. (1) **Manual hyperparameter transfer:** the best parameters found in notebook 3 are hand-copied into notebooks 4–5; a transcription slip would silently change results. (2) **The test set is also downsampled to 1:1** —

stated in the paper's Table 3 but easy to miss, and essential for interpreting their Table 7 numbers. (3) The exact subsampling procedure (stratification, seed) for their data reduction is not documented, so no third party can regenerate their exact training matrix.

Overall reproducibility of the work. *Directionally reproducible, not numerically reproducible.* With reimplementing effort I reproduced: the qualitative in-distribution ranking (DT/RF $\approx$ 99.7% > MLPs/SVM $\approx$ 96.5% > NB); 2 of 6 GridSearch optima exactly (SVM `C=100, gamma=1`; ANN `(50, ), tanh, alpha=1e-4`), with the other four differing only in regularisation magnitude; SVM's first place on the progressive test; and NB's near-total collapse (recall 0.008 vs. their 0.0003). I did **not** reproduce their ANN-vs-DNN ordering — under their own 1:1 protocol DNN outperforms ANN by 17 accuracy points, where the paper reports ANN ahead by 5. Because the paper reports single runs without variance, it is impossible to know whether this disagreement reflects their specific 11-feature set (ours overlaps 7/11), their subsample, or genuine instability of MLP training — which is itself a reproducibility finding: **the paper's model ranking is not stable under faithful re-implementation, but its methodological point survives.**

5. Experimental Results

Experiments performed. (1) Full pipeline reproduction: load and clean both datasets (~10% subsamples: 266,739 train / 106,906 test rows, 66 features), EDA, binary relabelling, 1:1 training-set balancing, feature selection to 11 features, k=5 GridSearchCV per model, k=5 CV as the in-distribution baseline, progressive evaluation on 2018 data. (2) **Real-prevalence evaluation** (84.1% benign test set — the deployment scenario the paper never reports). (3) **1:1-balanced progressive evaluation** replicating the paper's exact test protocol, enabling a like-for-like comparison with their Table 7. (4) **Overfitting-vs-concept-drift check:** comparing in-distribution CV (balanced 2017) against the balanced 2018 test — same class balance on both sides — so the drop isolates distribution change from base-rate effects. (5) Per-attack-type error analysis with recovered multiclass labels, plus FP/FN and threshold analysis.

Modifications introduced. 10% chunked subsampling with fixed seed; float32 downcasting; row/constant-column deduplication; four derived features (two entered the final 11); an extended metric suite (MCC, F_2 , ROC-AUC); keeping a real-prevalence copy of both training and test data; recovering per-attack-type labels for error analysis.

Models trained. All six of the paper's models (DT, RF, RBF-SVM, GaussianNB, ANN = 1×50 MLP, DNN = 3×50 MLP), GridSearch-tuned (k=5), fixed seed 42, trained on the balanced 96,572-row training matrix (SVM on a 20% subsample due to $O(n^2)$ fitting, mirroring the paper).

Evaluation metrics — definition and cybersecurity meaning. Accuracy $((TP+TN)/N)$ is reported for comparability with the paper but is misleading at 84% benign prevalence — always-predict-benign scores 84.1% with zero detection ability. Precision $(TP/(TP+FP))$ measures alert quality; low precision means SOC alert fatigue. Recall $(TP/(TP+FN))$ measures the fraction of real attacks caught; a false negative is a silent breach, generally the costlier error for an IDS. F1 (harmonic mean) weights the two errors equally — rarely correct for IDS — so I add F_2 , which weights recall double, encoding the asymmetric FN cost. **MCC** $((TP-TN-FP-FN)/\sqrt{((TP+FP)(TP+FN)(TN+FP)(TN+FN))})$ uses

all four confusion-matrix cells and is base-rate robust: it is the single most reliable number on this imbalanced test set (majority-class guessing gives $MCC = 0$). **ROC-AUC** (probability a random attack ranks above a random benign flow) is threshold-independent and separates a model's ranking quality from its (tunable) decision threshold. Regression metrics are inapplicable (classification task) and are omitted; accuracy is retained only because the paper's comparison requires it — each inclusion/exclusion is thereby justified.

Obtained results.

In-distribution ($k=5$ CV, balanced 2017): RF $99.73\% \pm 0.06$, DT $99.68\% \pm 0.07$, DNN $96.84\% \pm 0.32$, ANN $96.66\% \pm 0.17$, SVM $96.44\% \pm 0.29$, NB $84.44\% \pm 0.12$.

Progressive test, real prevalence (84.1% benign):

Model	Accuracy	Recall	MCC	ROC-AUC	FP count
SVM	0.8832	0.7448	0.6039	0.7929	8,157
DNN	0.8796	0.5802	0.5345	0.9075	5,749
RF	0.8626	0.1419	0.3378	0.5741	123
DT	0.8312	0.1433	0.1665	0.5522	3,503
ANN	0.8088	0.2735	0.2067	0.8244	8,112
NB	0.8084	0.0080	-0.0644	0.7111	3,648

Progressive test, 1:1-balanced (paper's protocol) vs. the paper's Table 7 accuracy: DT 0.552 vs. 0.594; RF 0.570 vs. 0.595; SVM **0.826 vs. 0.756**; NB 0.484 vs. 0.497; ANN **0.590 vs. 0.700**; DNN **0.757 vs. 0.652**. ranking: SVM > DNN > ANN; the paper's: SVM > ANN > DNN.

Matched-balance drop (CV $\rightarrow$ balanced 2018): DT -0.444, RF -0.427, ANN -0.376, NB -0.360 (ending below chance at 0.484), DNN -0.212, SVM -0.139 — while every model's CV std is $\leq 0.32\%$, the core evidence that the drop is concept drift, not overfitting.

Error analysis. The aggregate drop is driven by two large attack types: Brute Force-XSS ($n=10,489$; mean recall 0.234 across models; DT/RF at exactly 0) and DDoS-HOIC ($n=3,415$; mean recall 0.271). DDoS-LOIC-UDP sits at ~ 0.36 recall for every model — a feature-set ceiling, not a model failure. SVM and DNN have complementary blind spots (DNN misses HOIC, 0.003; SVM catches 81% of it), suggesting an ensemble. The missed XSS flows sit near the benign population mean on all 11 features (scaled values $\approx -0.5 \dots +0.3$): no threshold change recovers them; only new (application-layer) features would. On the FP side, RF's 123 false positives reflect passivity (14% recall), not precision; SVM trades the most FPs (8,157, $\sim 9\%$ of benign flows — a real alert-fatigue cost) for the highest recall; DNN offers the best FP/recall trade-off, and its ROC-AUC (0.9075) far above its default-threshold recall (0.58) makes it the clearest candidate for threshold tuning. Given the asymmetric cost of misses, a production deployment should tune thresholds down (favour recall) and treat the LOIC-UDP/XSS gaps as feature-engineering problems.

6. Conclusions

Key findings. (1) The paper's central methodological claim reproduces: progressive evaluation exposes a generalisation failure that same-distribution CV completely hides — models ranked by CV accuracy come out almost in reverse order of out-of-distribution merit. (2) SVM's first place reproduces; the ANN-over-DNN ranking reverses in run under the paper's own protocol, undermining the paper's specific deployment recommendation. (3) The "overfitting" label is contradicted by the CV-stability evidence; concept drift is the better explanation, and it demands a different mitigation (retraining on newer data, drift-robust features) than the one the paper's framing implies. (4) 1:1 balancing of the *test* set inflates and distorts reported performance: accuracy shifts of 22–32 points and ranking changes appear when the same models are scored at real prevalence.

Lessons learned. Accuracy on a balanced test set is a benchmark artifact, not a deployment estimate — MCC and per-class recall should lead IDS evaluation. Aggregate binary metrics hide the operative failures; per-attack-type analysis located the two classes driving the entire drop. RF-importance top-N selection does not remove even mathematically identical features — redundancy handling must be explicit. Binary label collapse destroys signal (our `feat_bwd_fwd_ratio` case). And single-run point estimates without variance make small ranking differences (like the paper's ANN-vs-DNN margin) unfalsifiable.

Strengths of the proposed solution. The progressive train-early/test-later design is simple, cheap, and genuinely diagnostic; the two-stage feature selection is transparent; the six-model comparison spans meaningfully different inductive biases; code and data are public.

Weaknesses. Metric set too narrow for an imbalanced problem; test-set balancing answers the wrong question; overfitting/drift conflation; no significance testing; no per-attack-type analysis; redundant features left in the final set; manual hyperparameter transfer as a reproducibility hazard.

Suggestions for future improvements. Evaluate at real prevalence with MCC/F₂/ROC-AUC and per-type recall; repeat across seeds/subsamples with confidence intervals; add an in-distribution stability check before attributing drops to overfitting; correlation-filter before importance ranking; explore class weights/SMOTE/threshold tuning instead of discarding 64% of training data; test an SVM+DNN ensemble (complementary blind spots); add application-layer features for the flow-invisible attack types; extend to a third, later dataset to measure drift as a function of time gap.

7. Executive Summary

This project reproduces and critically evaluates Chua & Salam (2023, *Symmetry* 15, 1251), which proposes *progressive evaluation* for ML-based intrusion detection: train six classifiers (DT, RF, SVM, NB, ANN, DNN) on CIC-IDS2017 and test them on the temporally later CSE-CIC-IDS2018, treating the performance drop as a measure of overfitting. The paper reports that DT/RF collapse while SVM and ANN resist the drop, and recommends ANN for long-term deployment.

We rebuilt the pipeline end-to-end in a Colab notebook backed by tested Python modules: chunked 10% loading of both datasets; EDA establishing real class imbalance (~82–84% benign), heavy-tailed feature distributions, and a Spearman-based redundancy scan (99 pairs $|\rho| > 0.90$, ten exact duplicates); reproduction of the authors' preprocessing (binary labels, 1:1 downsampling, RF-importance + brute-force selection to 11 features, 7/11 overlapping theirs) plus four derived features, two of which entered the final set; GridSearchCV tuning (2/6 optima match the paper exactly); and evaluation extended beyond the paper's Accuracy/Precision/Recall/F1 with MCC, F_2 , and ROC-AUC, at both real prevalence and the paper's 1:1 protocol, followed by per-attack-type error analysis.

Three of the paper's claims were confirmed: progressive evaluation exposes failure that CV hides (all models lose 14–44 points under matched class balance); SVM is the most drift-resistant model (recall 0.745, MCC 0.604); NB collapses (recall 0.008, MCC negative). Two were not. First, the ANN ranking reverses: DNN beats ANN on every generalisation metric, including a 17-point accuracy gap (0.757 vs. 0.590) under the paper's own balanced protocol — the deployment recommendation does not survive reproduction. Second, the "overfitting" diagnosis fails a direct test the paper never ran: every model's $k=5$ CV standard deviation is $\leq 0.32\%$ (DT/RF lowest of all), so no model shows within-distribution instability; the drop is concept drift, which calls for retraining and drift-robust features rather than regularisation. I further showed that the paper's 1:1 test balancing inflates results — DT/RF/NB accuracy falls 22–32 points at real prevalence — and that two attack types (Brute Force-XSS, DDoS-HOIC) drive most of the aggregate drop, with some flows provably indistinguishable from benign traffic on the chosen 11 features.

Bottom line: the paper's method is valuable and its headline result reproduces; its causal explanation and its model recommendation do not. Progressive evaluation should become standard practice for IDS research — paired with real-prevalence metrics and a drift-vs-overfitting check.

8. Summing It Up

The problem: whether ML-based IDS models that excel in-distribution keep detecting attacks on future, shifted traffic — and whether standard single-dataset, class-balanced evaluation can answer that.

The selected article: Chua & Salam (2023), *Evaluation of Machine Learning Algorithms in Network-Based Intrusion Detection Using Progressive Dataset*, Symmetry 15(6), 1251 — chosen because it clearly defines the problem, proposes a concrete solution, and publishes both code and data.

The dataset: CIC-IDS2017 (train) and CSE-CIC-IDS2018 (progressive test), ~10% subsamples, binary-relabelled, training set 1:1-balanced to mirror the paper, with real-prevalence copies kept for critique.

The methodology: full pipeline reproduction (loading → EDA → feature engineering → 6-model GridSearchCV training → multi-metric evaluation → per-attack-type error analysis) plus two original extensions: a matched-class-balance overfitting-vs-drift check, and a real-prevalence vs. 1:1 comparison that makes the paper's Table 7 numbers directly comparable to ours.

Main findings of the reproduction: SVM's drift resistance reproduces; ANN's claimed second place does not — DNN wins by 17 points under the paper's own protocol; NB's collapse reproduces; DT/RF track each other closely in both studies.

Were the author's claims supported? Partially. The methodological contribution is supported. The model ranking (SVM > ANN > DNN) is not — ours is SVM > DNN > ANN. The overfitting explanation is not supported once CV stability is examined.

Most important insights: balanced-test accuracy systematically misrepresents deployment behaviour; overfitting and drift share a symptom but need different fixes, so conflating them recommends the wrong mitigation; binary label collapse discards real per-attack-type signal.

Do I recommend this project for similar problems? Yes, with modifications: adopt progressive evaluation as a standard diagnostic, but pair it with an in-distribution stability check before naming the failure mode, and always report real-prevalence metrics alongside any balanced protocol.

Final conclusion: the paper's diagnosis (models degrade across the 2017→2018 shift) is correct and reproducible; its causal explanation (overfitting) and its specific recommendation (ANN) are not supported by a faithful reproduction — a result that strengthens, rather than weakens, the case for the paper's own progressive-evaluation methodology.